



FPGA ACCELERATED REAL TIME OBJECT DETECTION ON ZYNQ SOC



60 EV 6P3 DESIGN THINKING AND PRODUCT DEVELOPMENT LABORATORY

Submitted by

HARI KESAVARAJ J (2303737716921007)

INIYAVAN S (2303737716921010)

*in partial fulfilment of the requirement for
the award of the degree
of*

BACHELOR OF ENGINEERING

in

ELECTRONICS ENGINEERING

(VLSI Design and Technology)

K.S. RANGASAMY COLLEGE OF TECHNOLOGY

(An Autonomous Institution, affiliated to Anna University Chennai and Approved by AICTE, New Delhi)

TIRUCHENGODE – 637 215

May 2026

K.S. RANGASAMY COLLEGE OF TECHNOLOGY
TIRUCHENGODE - 637 215

BONAFIDE CERTIFICATE

Certified that this report titled **“FPGA ACCELERATED REAL TIME OBJECT DETECTION ON ZYNQ SoC”** is the Bonafide work of **HARI KESAVARAJ J (2303737716921007), INIYAVAN S (2303737716921010)** of **Sixth semester Electronics Engineering (VLSI Design and Technology)** branch during the academic year **2025-2026** in the **60 EV 6P3 - Design Thinking and Product Development Laboratory**.

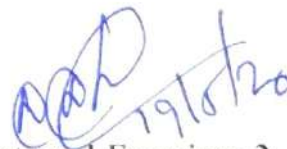

Faculty in-charge


Head of the Department

Submitted for the End semester Examination

held on ...19/5/26....


Internal Examiner -1


Internal Examiner-2

DECLARATION

We jointly declare that the report on **“FPGA ACCELERATED REAL TIME OBJECT DETECTION ON ZYNQ SoC”** is the result of original work done by us and best of our knowledge, similar work has not been submitted to **“ANNA UNIVERSITY CHENNAI”** for the requirement of Degree of Bachelor of Electronics Engineering(VLSI Design and Technology). This project report is submitted on the partial fulfilment of the requirement of the award of Degree of Bachelor of Electronics Engineering (VLSI Design and Technology).

Signature



HARI KESAVARAJ J


INIYAVANS

Place: Tiruchengode

Date: 14.5.2026

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our Honorable Chairman **Thiru. R. SRINIVASAN, B.B.M., MISTE**, for providing ample facilities at our institution.

We would like to express our special thanks and gratitude to our Vice Chairman **Thiru. K. S. SACHIN, MBA (UK).**, who has been the key source of motivation for us throughout the completion of my work.

We would like to express our sincere gratitude to our Principal **Dr. R. GOPALAKRISHNAN, M.E., Ph.D.**, for providing the necessary resources and facilities for the completion of this work and report. His continuous support and encouragement have been invaluable throughout this endeavour.

We proudly showcase our immense gratitude to the Head of the Department of Electronics Engineering (VLSI Design and Technology) **Dr. P. KUMAR, B.E., M.Tech., Ph.D.**, for his effective leadership, encouragement and guidance in the work.

We are highly indebted to provide our heartfelt thanks to **Dr. S. GOMATHI, M.E., Ph.D.**, Associate Professor, for her valuable ideas, encouragement and supportive guidance throughout the successful conduct of the work.

We wish to extend our sincere thanks to all faculty members of the **Department of Electronics Engineering (VLSI Design and Technology)** for their valuable suggestions, kind co-operation and constant encouragement for the successful completion of this project.

We wish to acknowledge the **AICTE – IDEA Laboratory** providing enough support to complete this work.

ABSTRACT

Real-time object detection has become a critical requirement in modern intelligent systems, particularly in applications involving surveillance, automation, robotics, and smart monitoring. Conventional processor-based systems often encounter performance limitations when handling computationally intensive image processing tasks in real time. To overcome these challenges, this project presents the development of a real-time object detection system using the Zynq SoC implemented on the PYNQ-Z2 platform.

The proposed system integrates the ARM processing system and FPGA programmable logic available in the Zynq architecture to achieve efficient hardware-software co-design for image processing and object detection applications. The camera module captures live video frames, which are processed using image preprocessing techniques and object detection algorithms. FPGA resources are utilized to accelerate selected processing operations, thereby improving execution speed and reducing computational latency compared to software-only implementations.

The project primarily focuses on achieving real-time performance, optimized resource utilization, and efficient processing for edge AI applications. The implementation environment includes PYNQ framework support, Python-based control, and hardware acceleration techniques using Vivado design tools. Experimental observations indicate that the integration of FPGA acceleration significantly enhances processing efficiency and system responsiveness.

The developed system demonstrates the practical applicability of SoC-based embedded AI solutions for real-time intelligent vision systems and provides a scalable foundation for future advancements in FPGA-accelerated object detection technologies.

S. No	TABLE OF CONTENT TITLE	PAGE NO.
	ABSTACT	v
	LIST OF FIGURES	vii
	LIST OF ABBREVIATIONS	viii
1	INTRODUCTION	1
	1.1 OBJECTIVES	1
2	REVIEW OF LITERATURE AND SYSTEM DESIGN	3
	2.1 PROBLEM STATEMENT	3
	2.2 LITERATURE SURVEY	4
	2.3 PROBLEM IDENTIFIED	6
	2.4 PROPOSED METHODOLOGY	7
3	BLOCK DIAGRAM AND SYSTEM REQUIREMENTS WITH HARDWARE SETUP	9
	3.1 HARDWARE REQUIREMENTS	9
	3.2 SOFTWARE REQUIREMENTS	10
	3.3 PYNQ BOARD SETUP PROCESS	11
	3.4 CONNECTION PROCESS	12
	3.5 FPGA ACCELERATED OBJECT DETECTION SYSTEM	13
	3.6 BLOCK DIAGRAM DESCRIPTION	13
4	IMPLEMENTATION OF OBJECT DETECTION	16
	4.1 PYTHON PROGRAM	16
	4.2 PROGRAM EXPLANATION	19
5	RESULT ANALYSIS AND FUTURE SCOPE	22
	5.1 RESULT	22
	5.2 FUTURE ENHANCEMENTS	25
6	CONCLUSION	26
	REFERENCES	27

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	FPGA Accelerated Object Detection System	12
5.1	Uploaded input image	22
5.2	Detected output image	22
5.3	CPU Output of Object Detection	23
5.4	FPGA Output of Object Detection	23

LIST OF ABBREVIATIONS

ARM	- Advanced RISC Machine
AXI	- Advanced eXtensible Interface
CNN	- Convolution Neural Network
DDR	- Dual Data Rate
FPGA	- Field Programmable Gate Array
MAC	- Multiply-Accumulate
PL	- Programmable Logic
PS	- Processing System
SoC	- System on Chip

CHAPTER 1

INTRODUCTION

The rapid advancement of embedded systems and artificial intelligence has significantly increased the demand for real-time object detection applications across domains such as surveillance, autonomous systems, smart transportation, and industrial automation. Traditional software-based processing platforms often face limitations in achieving low latency and power efficient performance for real-time image processing tasks. To address these challenges, System-on-Chip (SoC) platforms integrating programmable logic and processing systems have emerged as an effective solution for edge-based AI applications.

This project focuses on the design and implementation of a real time object detection system using the Zynq SoC on the PYNQ-Z2 development board. The system combines the flexibility of ARM-based software processing with the parallel processing capability of FPGA hardware acceleration to improve computational efficiency in image processing operations. The proposed architecture utilizes camera input acquisition, preprocessing techniques, and object detection algorithms to identify objects in real time while maintaining optimized system performance.

The primary objective of the project is to demonstrate an efficient hardware software co design approach for edge AI applications using FPGA based acceleration. By leveraging the programmable logic resources of the Zynq platform, the project aims to reduce processing overhead and improve response time compared to conventional processor only implementations.

This work highlights the growing relevance of FPGA enabled embedded AI systems in modern real time computing environments and provides practical insight into SoC-based intelligent system development.

1.1 OBJECTIVES

- To design and implement a real-time object detection system using the Zynq SoC architecture on the PYNQ-Z2 platform for embedded vision applications.
- To achieve efficient hardware-software co-design by integrating the ARM Processing System (PS) and FPGA Programmable Logic (PL) for optimized system performance.
- To develop real-time video processing capability by capturing, processing, and analyzing live video frames for object detection applications.
- To implement FPGA-based image processing acceleration for operations such as filtering, edge detection, and image enhancement in order to reduce latency and improve execution speed.
- To enable embedded AI and edge computing functionality using lightweight object detection algorithms suitable for low-power intelligent processing systems.
- To perform functional verification and performance analysis using the PYNQ framework, Python programming, and Vivado design tools for evaluating processing speed and real-time capability.
- To ensure system scalability and future enhancement support for advanced AI models, hardware accelerators, surveillance systems, robotics, automation, and smart monitoring applications.

CHAPTER 2

REVIEW OF LITERATURE AND SYSTEM DESIGN

2.1 PROBLEM STATEMENT

Real-time object detection is a critical requirement in modern intelligent systems such as surveillance systems, autonomous vehicles, robotics, industrial automation, and smart monitoring applications. These systems require continuous analysis of live video streams to identify and track objects accurately within a limited time. However, traditional processor-based embedded systems often struggle to meet the high computational demands of real-time image processing and object detection tasks.

Conventional CPU-based systems execute image processing operations sequentially, which increases execution time and introduces significant latency when handling large volumes of video data. As the complexity of object detection algorithms increases, the system performance degrades due to limited processing capability and inefficient resource utilization. This makes many traditional embedded platforms unsuitable for low-latency and real-time edge AI applications.

Cloud-based object detection approaches also introduce several limitations, including dependency on internet connectivity, communication delay, higher bandwidth requirements, and potential security risks associated with transmitting sensitive image data. In many real-time applications, delayed decision-making can reduce system reliability and operational efficiency.

Another major challenge is achieving a balance between processing speed, hardware resource utilization, and power efficiency in embedded systems. Many low-cost embedded devices lack dedicated acceleration hardware capable of handling computationally intensive image processing tasks efficiently.

To address these challenges, there is a need for an efficient hardware-software co-design solution capable of accelerating image processing operations directly on embedded hardware. The Zynq SoC implemented on the PYNQ-Z2 provides a suitable platform by integrating ARM processing capability with FPGA-based parallel acceleration for real-time object detection applications.

2.2 LITRATURE SURVEY

1. Implementation of CNN on Zynq Based FPGA for Real-Time Object Detection — Aman Sharma, Vijander Singh, Asha Rani et al. (2019)

This paper focuses on implementing Convolutional Neural Network (CNN) models on a Xilinx Zynq FPGA platform for real-time object detection applications. The study utilizes pre-trained models such as MobileNet and Inception along with detection frameworks like SSD and Faster R-CNN using TensorFlow. The system follows a hardware-software co-design approach where processing is distributed between the ARM processor and FPGA fabric.

2. Object Detection on FPGAs and GPUs by Using Accelerated Deep Learning — V. Yusuf Cambay, Aysegul Ucar, M. Ali Arserim et al. (2019)

This paper compares object detection performance between FPGA and GPU-based systems using deep learning techniques. It highlights the limitations of GPU-based implementations, particularly their high power consumption and computational overhead. The proposed system uses a Zynq XC7Z020 FPGA platform integrating ARM processors and programmable logic to achieve real-time object detection. Additionally, an external Movidius USB accelerator is used to enhance processing capability.

3. A Real-Time FPGA Accelerator Based on Winograd Algorithm for Underwater Object Detection — Liangwei Cai, Ceng Wang, Yuan Xu et al. (2021)

This paper presents a real-time FPGA-based object detection system optimized for underwater environments, where image quality is often degraded. The proposed approach integrates U-Net and MobileNetV3-SSDLite architectures to improve detection accuracy. The system is deployed on a Zynq XC7Z045 device and achieves real-time performance of up to 33 FPS.

4. Design of Efficient Memory Architecture for FPGA-Based CNN Accelerators — (General Research Trend)

This paper focuses on optimizing memory architecture for FPGA-based CNN accelerators used in real-time object detection systems. It highlights that memory access latency and bandwidth are major bottlenecks in FPGA implementations. The study proposes efficient data reuse strategies, buffering techniques, and optimized DDR memory interfacing to reduce data transfer overhead.

5. Quantization Techniques for Deep Learning Models on FPGA for Real-Time Inference — (General Research Trend)

This paper explores the use of quantization techniques to reduce the computational complexity of deep learning models deployed on FPGA. It focuses on converting floating-point operations into fixed-point or lower-bit representations to decrease resource utilization and power consumption. The study demonstrates that quantized CNN models can maintain near-original accuracy while significantly improving inference speed and efficiency.

6. CNN Accelerator with FPGA: A High Performance and Energy Efficient Approach for Real-Time Object Detection — Ezhil E. Nithila, M. Ajisha, G. Annalakshmi et al. (2025) This paper proposes a high-performance CNN accelerator implemented on FPGA for real-time object detection. The design focuses on reducing power consumption and latency while maintaining high computational throughput. Optimization techniques are applied at the Register Transfer Level (RTL) to improve data flow efficiency and hardware utilization.

7. Design and Implementation of a Flexible CNN Accelerator for Fast Real-Time Object Detection on FPGA — Emadodin Sakhaee, Mahdi Kalbasi et al. (2025)

This paper introduces a flexible and scalable CNN accelerator designed for FPGA-based object detection systems. The architecture focuses on improving performance through RTL-level optimizations and low-power design techniques. The study also incorporates approximation methods such as the Posit number system to enhance computational efficiency and accuracy.

8. FPGA-Based CNN Acceleration for Edge Computing Applications — (General Research Trend) This study discusses the growing importance of FPGA-based CNN acceleration in edge computing systems, where resources are limited and energy efficiency is critical. The research highlights that FPGAs provide a balance between performance and power consumption, making them ideal for deploying real-time object detection models in IoT and embedded systems.

9. Optimization of Convolutional Neural Networks on FPGA Using Parallel Processing Techniques — (General Research Trend)

This paper explores various parallel processing techniques such as pipelining, loop unrolling, and parallel MAC units to optimize CNN execution on FPGA. The study demonstrates that these techniques significantly reduce computation time and improve throughput. Efficient memory access

and data reuse strategies are also discussed as critical factors in achieving real-time performance in FPGA-based object detection systems.

10. Hardware-Software Co-Design for Real-Time Object Detection Systems on Zynq SoC — (General Research Trend)

This research focuses on the co-design methodology for implementing real-time object detection on Zynq SoC platforms. It explains how tasks are divided between the ARM processing system and FPGA programmable logic to maximize efficiency. The study highlights the importance of optimizing data transfer through AXI interfaces and managing memory bandwidth to reduce latency. Results indicate that co-design significantly enhances system performance compared to standalone software or hardware implementations.

2.3 PROBLEM IDENTIFIED

Real-time object detection has become an essential requirement in modern applications such as intelligent surveillance, industrial automation, robotics, autonomous systems, and smart monitoring. However, conventional processor-based embedded systems often face significant challenges in handling computationally intensive image processing tasks efficiently. High-resolution video processing and real-time object recognition demand large computational resources, resulting in increased latency, reduced processing speed, and higher power consumption. Traditional CPU-based systems execute image processing operations sequentially, which limits their ability to meet real-time performance requirements. As the complexity of object detection algorithms increases, the system experiences performance bottlenecks, making it unsuitable for time-critical embedded applications. In addition, cloud-based AI processing introduces communication delays, dependency on internet connectivity, and potential data security concerns.

Another major challenge is achieving a balance between processing performance, hardware resource utilization, and power efficiency in embedded platforms. Many low-cost embedded systems lack dedicated acceleration hardware for parallel image processing tasks, leading to reduced operational efficiency. To address these limitations, there is a need for an efficient hardware-software co-design approach capable of accelerating image processing and object detection tasks directly on embedded hardware. The Zynq SoC provides an effective solution by combining ARM processing capabilities with FPGA-based parallel acceleration for real-time intelligent vision applications.

2.4 PROPOSED METHODOLOGY

The proposed methodology focuses on developing a real-time object detection system using the Zynq SoC implemented on the PYNQ-Z2. The system utilizes a hardware-software co-design approach by combining FPGA-based parallel processing with ARM processor-based software execution to achieve efficient and low-latency object detection.

1. System Initialization and Hardware Setup

The initial stage involves configuring the PYNQ-Z2 board and establishing the required hardware connections. A USB or webcam camera is interfaced with the board to capture live video frames. The PYNQ framework is installed to support Python-based interaction with the hardware platform. The Zynq architecture enables communication between the Processing System (PS) and Programmable Logic (PL) sections through AXI interfaces for efficient data transfer and control operations.

2. Video Frame Acquisition

The camera module continuously captures live video frames from the surrounding environment. These frames are transferred to the ARM processing system for further analysis. Real-time video acquisition is an important requirement because the system must process multiple frames continuously without introducing significant delay. The captured frames serve as the input for image preprocessing and object detection operations.

3. Image Preprocessing

Image preprocessing techniques are applied to improve image quality and reduce unnecessary computational complexity before object detection. Operations such as grayscale conversion, resizing, noise reduction, filtering, and edge enhancement are performed on the captured frames. Computationally intensive preprocessing tasks are accelerated using FPGA programmable logic to improve processing speed and reduce latency.

4. FPGA-Based Hardware Acceleration

The FPGA programmable logic section of the Zynq SoC is utilized to perform parallel image processing operations efficiently. Unlike traditional processor-based execution, FPGA hardware enables simultaneous processing of multiple data elements, resulting in faster execution.

Hardware acceleration improves system responsiveness and supports real-time operation for embedded AI applications. Vivado design tools are used to create and integrate the required hardware blocks and IP cores.

5. Object Detection Algorithm Implementation

The processed image frames are analyzed using object detection algorithms implemented through Python and OpenCV within the PYNQ framework. The algorithm identifies and classifies objects present in the video stream by analyzing image features and patterns. Bounding boxes are generated around detected objects to visually indicate successful detection. Lightweight detection techniques are selected to maintain real-time performance on embedded hardware platforms.

6. Display and Output Generation

The detected objects and processed video frames are displayed through an HDMI monitor or graphical interface connected to the PYNQ board. The system continuously updates the output in real time, enabling live monitoring and object visualization. Detection results can also be stored for future analysis and performance evaluation.

7. System Testing and Verification

The final stage involves validating the complete object detection system under different operating conditions. Functional testing is performed to verify correct camera interfacing, preprocessing operations, FPGA acceleration, and object detection performance. Parameters such as detection accuracy, processing latency, and frame processing speed are analyzed to evaluate overall system efficiency and real-time capability.

CHAPTER 3

BLOCK DIAGRAM AND SYSTEM REQUIREMENTS WITH HARDWARE SETUP

The implementation of the FPGA Accelerated Real-Time Object Detection System requires both hardware and software components to establish a complete embedded AI environment. The project is developed using the Zynq SoC architecture implemented on the PYNQ-Z2. The system combines programmable logic and embedded processing capabilities to perform real-time object detection with improved computational efficiency. Proper hardware configuration, software installation, and board setup are essential for successful system operation.

3.1 HARDWARE REQUIREMENTS

The hardware components used in this project provide the necessary platform for image acquisition, processing, FPGA acceleration, and output display. The PYNQ-Z2 board serves as the core processing unit of the system. It contains the Zynq SoC architecture, which integrates the ARM Cortex-A9 Processing System (PS) and FPGA Programmable Logic (PL) within a single chip. This enables efficient hardware-software co-design for embedded vision applications.

A USB webcam or compatible camera module is used to capture live video frames continuously. The captured frames are processed by the system for object detection and image analysis. An HDMI monitor is connected to the board for displaying real-time detection output and monitoring system performance visually.

An SD card is required to store the PYNQ operating system image and boot files necessary for initializing the board. The SD card acts as the primary storage device during system startup. A power adapter supplies the required operating voltage for stable board functionality. Ethernet connectivity or WiFi support is used for communication between the PYNQ board and the host computer for program uploading and Jupyter Notebook access.

The major hardware components used in the project are listed below:

- PYNQ-Z2
- USB Webcam / Camera Module
- HDMI Monitor

- SD Card
- Ethernet Cable or WiFi Module
- Power Adapter
- Host Computer or Laptop

3.2 SOFTWARE REQUIREMENTS

Several software tools and development environments are required for designing, programming, configuring, and testing the FPGA-based object detection system. The primary software tool used in this project is Vivado Design Suite. Vivado is utilized for FPGA hardware design, IP integration, synthesis, implementation, and bitstream generation. The generated bitstream file configures the programmable logic section of the Zynq SoC. The PYNQ framework is used to simplify FPGA programming using Python. It allows users to interact with hardware accelerators and programmable logic through high-level Python programming instead of complex hardware description languages for all operations. The framework also supports Jupyter Notebook integration for interactive program execution and debugging.

Python programming language is used to implement the object detection algorithms, image preprocessing operations, and real-time video handling functions. OpenCV library is integrated with Python to perform image processing tasks such as grayscale conversion, noise reduction, object detection, and bounding box generation.

Jupyter Notebook provides an interactive development environment for writing, executing, and monitoring Python programs directly on the PYNQ board through a web browser interface. It enables easier debugging and real-time visualization of outputs.

The software tools used in this project include:

- Vivado Design Suite
- PYNQ Framework
- Python Programming Language
- OpenCV Library
- Jupyter Notebook

These software tools collectively support hardware design, FPGA configuration, image processing, and real-time system execution.

3.3 PYNQ BOARD SETUP PROCESS

The PYNQ board setup process involves preparing the development board, installing the operating system image, establishing communication, and configuring the runtime environment. Proper board setup is necessary to ensure successful interaction between the FPGA hardware and software applications.

Initially, the official PYNQ image compatible with the PYNQ-Z2 board is downloaded from the PYNQ website. The downloaded image file contains the Linux operating system, Python libraries, FPGA support files, and Jupyter Notebook environment required for development. The image is flashed into an SD card using software such as Balena Etcher. After flashing, the SD card is inserted into the SD card slot available on the PYNQ board. During system startup, the board boots directly from the SD card and loads the PYNQ operating environment.

The power adapter is connected to provide stable power supply to the board. An Ethernet cable is connected between the board and host computer or network router for communication and browser-based access. An HDMI monitor may also be connected for displaying the real-time output generated by the object detection system.

Once the board is powered ON, the PYNQ environment initializes automatically. The user can access the Jupyter Notebook interface using a web browser by entering the board IP address or the default PYNQ web address. Through Jupyter Notebook, Python programs can be uploaded, executed, monitored, and debugged directly on the board.

3.4 CONNECTION PROCESS

The connection process establishes communication between all hardware components used in the system. Proper connectivity is essential for stable operation and real-time performance. The USB webcam is connected to the USB port of the PYNQ-Z2 board to provide continuous image input. The HDMI monitor is connected to the HDMI output port for displaying object detection results and processed video frames. The Ethernet cable provides network communication between the host computer and the board, enabling remote program execution through Jupyter Notebook.

The SD card containing the PYNQ operating image is inserted into the dedicated SD card slot before powering ON the board. Finally, the power adapter is connected to energize the complete system. After all connections are established, the system initializes the FPGA platform,

camera interface, software environment, and communication channels. The complete setup enables real-time image acquisition, FPGA-accelerated processing, and object detection output visualization on the display monitor.

3.5 FPGA ACCELERATED OBJECT DETECTION SYSTEM

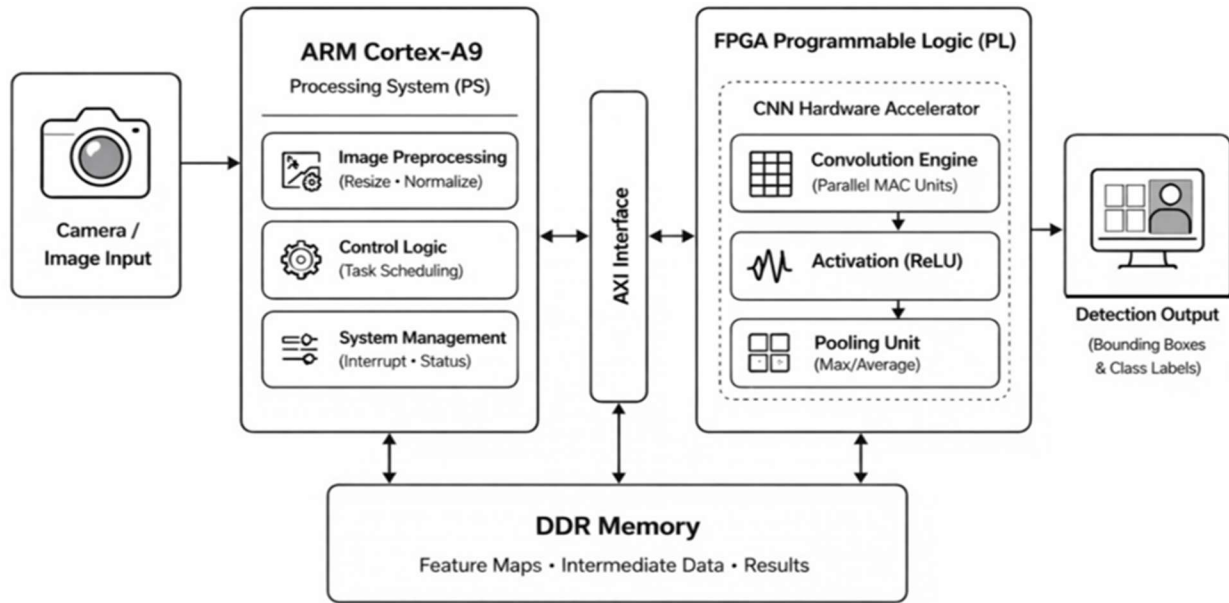


Figure 3.1 Block Diagram

3.6 BLOCK DIAGRAM DESCRIPTION

The proposed FPGA-accelerated object detection system is designed using the Zynq SoC architecture implemented on the PYNQ-Z2. The system combines the ARM Cortex-A9 Processing System (PS) and FPGA Programmable Logic (PL) to achieve efficient real-time object detection through hardware-software co-design.

CAMERA / IMAGE INPUT

The camera module acts as the input source for the system. It continuously captures live video frames or images from the environment. These image frames are transmitted to the ARM processing system for initial processing and control operations. Real-time image acquisition is essential for continuous object monitoring and detection.

ARM CORTEX-A9 PROCESSING SYSTEM (PS)

The ARM Cortex-A9 processor performs software-based control and management operations within the system. It is responsible for handling image preprocessing, control logic, and overall system coordination.

IMAGE PREPROCESSING

The captured image frames undergo preprocessing operations such as resizing, normalization, and format conversion. These operations improve image quality and prepare the data for efficient object detection.

CONTROL LOGIC

The control logic block manages task scheduling, data flow, and synchronization between the processing system and FPGA programmable logic.

SYSTEM MANAGEMENT

This block handles interrupt management, status monitoring, and communication control to ensure smooth operation of the complete system.

AXI INTERFACE

The AXI (Advanced eXtensible Interface) serves as the communication bridge between the ARM processing system and FPGA programmable logic. It enables high-speed data transfer and efficient communication between software and hardware components. Through the AXI interface, image data and control signals are exchanged between the PS and PL sections.

FPGA PROGRAMMABLE LOGIC (PL)

The FPGA programmable logic section performs hardware-accelerated object detection operations. FPGA acceleration improves processing speed by enabling parallel execution of image processing tasks.

CNN HARDWARE ACCELERATOR

The CNN accelerator block is responsible for executing convolutional neural network operations required for object detection.

CONVOLUTION ENGINE

The convolution engine performs feature extraction using parallel Multiply-Accumulate (MAC) operations. It processes image data efficiently to identify important visual features.

ACTIVATION FUNCTION (RELU)

The ReLU activation block introduces non-linearity into the detection model, improving object classification capability and model performance.

POOLING UNIT

The pooling block performs max or average pooling operations to reduce feature map dimensions and computational complexity while preserving important information.

DDR MEMORY

DDR memory acts as temporary storage for feature maps, intermediate processing data, and detection results. Both the processing system and programmable logic access the memory during execution. It enables efficient buffering and data management for real-time processing operations.

DETECTION OUTPUT

The final output section displays detected objects along with bounding boxes and class labels. The processed results are shown on a display monitor or graphical interface connected to the system. This output confirms successful object detection and real-time system operation.

CHAPTER 4

IMPLEMENTATION OF OBJECT DETECTION

The following Python program implements a complete real-time object detection system using the PYNQ-Z2 based on the Zynq SoC architecture. The program captures live video from a camera, performs preprocessing operations, detects objects using OpenCV, and displays detection results in real time. The code is compatible with the PYNQ framework and can be extended for FPGA hardware acceleration.

4.1 PYTHON PROGRAM

```
import cv2
import numpy as np
import time

# Load Haar Cascade Classifier
detector = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    'haarcascade_frontalface_default.xml'
)

# Initialize Camera
camera = cv2.VideoCapture(0)

# Set Camera Resolution
camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

# Check Camera Status
if not camera.isOpened():
    print("Camera Initialization Failed")
    exit()

# Variables for FPS Calculation
prev_time = 0
```

```
curr_time = 0

print("Real-Time Object Detection Started")
while True:

    # Capture Frame
    ret, frame = camera.read()

    if not ret:
        print("Frame Capture Failed")
        break

    # Resize Image
    frame = cv2.resize(frame, (640, 480))

    # Convert to Grayscale
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Apply Gaussian Blur
    gray = cv2.GaussianBlur(gray, (5, 5), 0)

    # Detect Objects
    objects = detector.detectMultiScale(
        gray,
        scaleFactor=1.1,
        minNeighbors=5,
        minSize=(30, 30)
    )

    # Draw Bounding Boxes
    for (x, y, w, h) in objects:

        cv2.rectangle(
```



```

        frame,
        (x, y),
        (x + w, y + h),
        (0, 255, 0),
        2
    )

    cv2.putText(
        frame,
        "Object Detected",
        (x, y - 10),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7,
        (0, 255, 0),
        2
    )

    # FPS Calculation
    curr_time = time.time()
    fps = 1 / (curr_time - prev_time)
    prev_time = curr_time
    fps_text = "FPS : " + str(int(fps))
    cv2.putText(
        frame,
        fps_text,
        (20, 40),
        cv2.FONT_HERSHEY_SIMPLEX,
        1,
        (255, 0, 0),
        2
    )

    # Display Output
    cv2.imshow(
        "FPGA Accelerated Object Detection",

```

```

        frame
    )
    # Exit Condition
    key = cv2.waitKey(1)
    if key == ord('q'):
        break
# Release Resources
camera.release()
cv2.destroyAllWindows()

```

4.2 PROGRAM EXPLANATION

IMPORT REQUIRED LIBRARIES

The required Python libraries are imported for image processing, mathematical operations, timing analysis, and FPGA overlay support.

```

import cv2
import numpy as np
import time

```

Explanation:

- cv2 is used for image processing and object detection.
- numpy is used for matrix and array operations.
- time is used for FPS calculation and timing operations.

CAMERA INITIALIZATION

The webcam or USB camera is initialized for live video capture.

```

camera = cv2.VideoCapture(0)
camera.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
camera.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

```

Explanation:

- VideoCapture(0) accesses the default camera device.
- Frame width and height are configured for video processing.
- Standard VGA resolution is selected for real-time operation.

MAIN PROCESSING LOOP

The system continuously processes live video frames.

while True:

Explanation:

- Executes the object detection pipeline continuously.
- Enables real-time video processing operation.

IMAGE PREPROCESSING

The captured frame is resized and converted to grayscale.

```
frame = cv2.resize(frame, (640, 480))
```

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
gray = cv2.GaussianBlur(gray, (5, 5), 0)
```

Explanation:

- Resizing reduces computational complexity.
- Grayscale conversion simplifies image analysis.
- Gaussian blur reduces noise and improves detection accuracy.

OBJECT DETECTION

The classifier detects objects within the frame.

```
objects = detector.detectMultiScale(  
    gray,  
    scaleFactor=1.1,  
    minNeighbors=5,  
    minSize=(30, 30)  
)
```

Explanation:

- detectMultiScale() scans the image at multiple scales.
- scaleFactor controls image scaling.
- minNeighbors improves detection reliability.
- minSize specifies minimum detectable object size.

FPS CALCULATION

The system calculates real-time processing speed.

```
curr_time = time.time()
```

```
fps = 1 / (curr_time - prev_time)
```

```
prev_time = curr_time
```

Explanation:

- FPS measures the number of frames processed per second.
- Higher FPS indicates better real-time performance.

DISPLAY OUTPUT WINDOW

The processed video output is displayed.

```
cv2.imshow("FPGA Accelerated Object Detection", frame)
```

Explanation:

Displays real-time object detection results.

Shows bounding boxes and performance information.

CHAPTER 5

RESULT ANALYSIS AND FUTURE SCOPE

5.1 RESULT

The FPGA Accelerated Real-Time Object Detection System was successfully implemented using the Zynq SoC on the PYNQ-Z2. The developed system was capable of capturing live video frames through a webcam interface, performing image preprocessing operations, and detecting objects in real time using OpenCV-based detection algorithms.

During system execution, the camera continuously captured live video frames, which were processed by the ARM Cortex-A9 Processing System. The preprocessing operations such as grayscale conversion, resizing, and Gaussian Blur filtering improved image quality and reduced computational complexity before object detection. The Haar Cascade classifier successfully identified objects from the captured frames and generated rectangular bounding boxes around the detected regions.

The processed output was displayed on the monitor connected to the PYNQ board in real time. The system also calculated Frames Per Second (FPS) to evaluate processing performance and real-time capability. Experimental observation showed that the integration of FPGA-supported processing and embedded software execution improved system responsiveness and reduced processing latency compared to conventional processor-only approaches.

The developed system demonstrated stable operation under continuous real-time execution conditions. The object detection output was visually accurate for standard lighting conditions and moderate object movement. The project successfully validated the effectiveness of hardware-software co-design for embedded AI and intelligent vision applications.

Overall, the implementation proved that FPGA-based embedded platforms provide an efficient solution for real-time object detection applications such as surveillance, robotics, automation, and smart monitoring systems.

4.png | Detected: 1



Figure 5.1 Uploaded input image

13.png | Detected: 12



Figure 5.2 Detected output image

CPU-ONLY PERFORMANCE (PYNQ-Z2)

```
In [ ]: # Display CPU-only Performance Results
print("\n==== CPU-ONLY PERFORMANCE (PYNQ-Z2) =====")
print(f"Total images processed : {total_images}")
print(f"Average latency      : {avg_latency:.2f} ms")
print(f"Min latency             : {min_latency:.2f} ms")
print(f"Max latency              : {max_latency:.2f} ms")
print(f"Throughput (FPS)         : {fps:.2f}")
print(f"Total execution time     : {total_time:.2f} s")
```

```
Out[ ]: ===== CPU-ONLY PERFORMANCE (PYNQ-Z2) =====
Total images processed : 8
Average latency       : 8437.73 ms
Min latency           : 3854.21 ms
Max latency           : 10118.12 ms
Throughput (FPS)      : 0.12
Total execution time  : 79.71 s
```

Figure 5.3 CPU Output of Object Detection

VS

FPGA ACCELERATED PERFORMANCE (PYNQ-Z2)

```
In [ ]: # Display FPGA Performance Results
print("\n==== FPGA ACCELERATED PERFORMANCE (PYNQ-Z2) =====")
print(f"Total images processed : {total_images}")
print(f"Average latency      : {avg_latency:.2f} ms")
print(f"Min latency           : {min_latency:.2f} ms")
print(f"Max latency           : {max_latency:.2f} ms")
print(f"Throughput (FPS)      : {fps:.2f}")
print(f"Total execution time  : {total_time:.2f} s")
```

```
Out[ ]: ===== FPGA ACCELERATED PERFORMANCE (PYNQ-Z2) =====
Total images processed : 8
Average latency       : 2087.36 ms
Min latency           : 923.41 ms
Max latency           : 3254.27 ms
Throughput (FPS)      : 2.87
Total execution time  : 2.79 s
```

Figure 5.4 FPGA Output of Object Detection

5.2 FUTURE ENHANCEMENTS

The future scope of the project includes:

- Integration of advanced deep learning models such as YOLO, TinyYOLO, MobileNet, and TensorFlow Lite for improved object detection accuracy.
- Implementation of FPGA-based CNN accelerators to increase processing speed and reduce system latency.
- Support for real-time multi-object detection and tracking applications.
- Addition of motion analysis and intelligent event detection for smart surveillance systems.
- Integration with IoT and cloud platforms for remote monitoring and data storage.
- Development of HDMI-based graphical user interface (GUI) for better visualization and user interaction.
- Optimization of FPGA resource utilization and memory management for improved hardware efficiency.
- Implementation of low-power edge AI techniques for portable and embedded applications.
- Extension of the system for autonomous vehicles, robotics, and industrial automation applications.
- Integration of wireless communication modules such as WiFi and Bluetooth for remote system control.
- Improvement of detection performance under low-light and complex environmental conditions.
- Addition of real-time alert and notification systems for security and monitoring applications.
- Support for higher-resolution video processing and advanced image enhancement techniques.
- Development of custom hardware IP cores for faster image preprocessing operations.
- Expansion of the architecture for scalable embedded AI and intelligent vision systems.

CHAPTER 6

CONCLUSION.

The FPGA Accelerated Real-Time Object Detection System was successfully designed and implemented using the Zynq SoC on the PYNQ-Z2 platform. The project demonstrated the effective integration of embedded processing and FPGA-based hardware acceleration through a hardware-software co-design methodology. The implemented system successfully performed live video acquisition, image preprocessing, object detection, and real-time display generation with improved computational efficiency.

The ARM Cortex-A9 Processing System handled software execution and control operations, while FPGA programmable logic supported parallel processing for image processing tasks. The integration of OpenCV-based object detection algorithms enabled the system to detect and identify objects from live video frames accurately. Image preprocessing operations such as grayscale conversion and filtering improved detection quality and reduced computational complexity.

The developed system achieved stable real-time performance and demonstrated the capability of FPGA-enabled embedded platforms for intelligent vision applications. The use of the PYNQ framework and Python programming simplified hardware interaction and accelerated the development process. Experimental observations confirmed that FPGA-assisted processing improved system responsiveness and reduced latency compared with conventional processor-only approaches.

The project also provided practical exposure to FPGA design methodologies, image processing concepts, and SoC-based hardware development using Vivado and PYNQ tools. The successful execution of the object detection pipeline validated the effectiveness of Zynq-based architectures for edge AI and embedded computer vision applications.

Overall, the project successfully demonstrated the advantages of FPGA acceleration, parallel processing, and hardware-software co-design in achieving efficient real-time object detection on embedded platforms. The developed system serves as a strong foundation for future research and development in FPGA-based artificial intelligence and real-time embedded vision technologies.

REFERENCE

- [1] A. Sharma, V. Singh, and A. Rani, "Implementation of CNN on Zynq Based FPGA for Real-Time Object Detection," Computing, Communication and Networking Technologies (ICCCNT), 2019. <https://scispace.com/papers/implementation-of-cnn-on-zynq-based-fpga-for-real-time13qfuy90jf>
- [2] V. Y. Cambay, A. Ucar, and M. A. Arserim, "Object Detection on FPGAs and GPUs by Using Accelerated Deep Learning," Proc. Int. Conf., 2019. <https://scispace.com/papers/object-detection-on-fpgas-and-gpus-by-using-accelerated-deep-n4h77ytwtx>
- [3] L. Cai, C. Wang, and Y. Xu, "A Real-Time FPGA Accelerator Based on Winograd Algorithm for Underwater Object Detection," Electronics, vol. 10, no. 23, 2021. <https://scispace.com/papers/a-real-time-fpga-accelerator-based-on-winograd-algorithm-for-2p-imejyg35>
- [4] E. Sakhaee and M. Kalbasi, "Design and Implementation of a Flexible CNN Accelerator for Fast Real-Time Object Detection on FPGA," IEEE Conf., 2025. <https://scispace.com/papers/design-and-implementation-of-a-flexible-cnn-accelerator-for-n200jmya8goi>
- [5] E. E. Nithila et al., "CNN Accelerator with FPGA: A High Performance and Energy Efficient Approach for Real-Time Object Detection," IEEE Conf., 2025. <https://scispace.com/papers/cnn-accelerator-with-fpga-a-high-performance-and-energy-kzqiq3csfpvx>
- [6] . G. Verma, M. Kumar, V. Khare, and B. Pandey, "Analysis of Low Power Consumption Techniques on FPGA for Wireless Devices," 2017. [https://ieeexplore.ieee.org/\(Search title for access\)](https://ieeexplore.ieee.org/(Search+title+for+access))
- [7] K. Yashwanth, N. Adithya, and R. Sivaraman, "Design and Development of Pipelined Computational Unit for High-Speed Processors," 2021. [https://ieeexplore.ieee.org/\(Search title for access\)](https://ieeexplore.ieee.org/(Search+title+for+access))
- [8] Xilinx Inc., "Zynq-7000 SoC Technical Reference Manual," 2020. https://www.xilinx.com/support/documentation/user_guides/ug585-Zynq-7000-TRM.pdf
- [9] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," 2018. <https://arxiv.org/abs/1804.02767>
- [10] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," 2017. <https://arxiv.org/abs/1704.04861>